

План демонстрации программного продукта

Тема ВКР: «Разработка системы управления контентом в сфере маркетинга на базе Strapi и Astro»

Назначение документа

Документ фиксирует план демонстрации разработанного программного продукта без внесения изменений в код проекта. План опирается на уже реализованный и подтвержденный baseline системы и предназначен для предзащиты ВКР.

Цель демонстрации

Показать, что разработанная система:

- реализует CMS-first подход к управлению контентом;
- поддерживает отдельные контуры публичной витрины и административного управления;
- обеспечивает локализацию, предварительный просмотр, базовый SEO-контур и формы взаимодействия с пользователем;
- имеет воспроизводимый публикационный контур `publish` → `webhook` → `rebuild`.

Формат демонстрации

Рекомендуемая длительность: 10 -- 15 минут.

Основной формат:

1. краткое введение в архитектуру;
2. показ административного контура Strapi;
3. показ публичной витрины Astro;
4. показ preview-сценария;
5. краткий разбор эксплуатационного и публикационного контура;
6. фиксация ограничений текущей версии.

Среда демонстрации

Требуемые сервисы

- frontend runtime: `http://localhost:4321`;
- CMS runtime: `http://localhost:1337`;
- при необходимости build-preview: `http://localhost:4322`.

Минимальная проверка перед показом

Перед демонстрацией рекомендуется убедиться, что:

- открывается главная страница `/ru/`;
- открывается английская версия `/en/`;
- доступна Strapi admin panel;
- работает хотя бы один preview-сценарий для page;
- в CMS есть опубликованные записи для page, article, project, vacancy;
- в CMS есть хотя бы один draft для показа preview;
- заранее известны логины ролей, если планируется показ разделения доступа.

Что не требуется

- не требуется вносить изменения в код проекта;
- не требуется live-доработка данных во время выступления;
- не требуется запуск полного production-пути с внешним Vercel rebuild.

Сценарий демонстрации

1. Введение и постановка задачи

Время: 1 -- 2 минуты.

Что показать и проговорить:

- тему ВКР и назначение системы;
- проблему кодо-центричного управления маркетинговым сайтом;
- идею разделения на Strapi как контентное ядро и Astro как публичную витрину.

Ожидаемый результат: комиссия понимает, зачем в проекте одновременно нужны CMS, публичный frontend, preview и публикационный контур.

2. Архитектура и контуры системы

Время: 1 -- 2 минуты.

Что показать:

- краткую схему взаимодействия Strapi → API → Astro;
- два основных контура:
 - маркетинговый: `global`, `home-page`, `page`, `article`, `author`, `project`, `lead-submission`;
 - карьерный: `vacancy`, `industry`, `job-role`, `vacancy-application`.

Что проговорить:

- роли `administrator`, `marketer/content-manager`, `editor`, `hr`;
- принцип, что публичная витрина не пишет в Strapi напрямую из браузера.

Ожидаемый результат: показано, что система спроектирована как разделенная управляемая платформа.

3. Демонстрация CMS-контура

Время: 2 -- 3 минуты.

Что показать в Strapi:

- список ключевых сущностей;
- структуру `home-page` или `page`;
- наличие `Dynamic Zone` у страниц;
- SEO-поля у страницы или детальной сущности;
- наличие опубликованного и чернового контента;
- при возможности — разделение ролей или описание того, кто что редактирует.

Что проговорить:

- `editor` готовит черновик, но не публикует;
- `marketer/content-manager` публикует маркетинговый контент;
- `hr` управляет вакансиями и откликами.

Ожидаемый результат: показано, что контент редактируется в CMS, а не в исходном коде frontend.

4. Демонстрация публичной витрины

Время: 3 -- 4 минуты.

Что открыть в браузере:

1. `/ru/` — главная страница;
2. `/en/` — английская версия главной;
3. representative page, например `/ru/cms-first-platform/`;
4. список или detail статьи `/ru/articles/ ...`;
5. список или detail проекта `/ru/projects/ ...`;
6. `/vacancies/` и detail вакансии.

Что проговорить:

- storefront-core работает в локалях `ru/en`;
- страницы, статьи и проекты приходят из CMS;
- вакансии вынесены в отдельный прикладной модуль;
- frontend выполняет роль публичной витрины и использует данные Strapi.

Ожидаемый результат: комиссия видит, что система реально публикует контентные разделы, а не ограничивается только админкой.

5. Демонстрация preview-сценария

Время: 2 минуты.

Что показать:

- preview action для черновой page или другой сущности;
- переход через `/api/preview`;
- открытие маршрута `/preview/ ...`;
- отличие preview от обычной публичной страницы.

Что проговорить:

- preview работает без раскрытия публичного draft API;
- draft-чтение выполняется через `PREVIEW_SECRET`;
- preview-страницы получают `noindex`.

Ожидаемый результат: показан один из ключевых инженерных результатов — безопасный предпросмотр черновика.

6. Демонстрация форм взаимодействия

Время: 1 -- 2 минуты.

Безопасный вариант показа:

- открыть публичную форму лида или отклика;
- показать обязательные поля;
- продемонстрировать валидацию на невалидных данных;
- при необходимости кратко показать, что отправка идет через server-side route, а не напрямую в `Strapi`.

Если нежелательно изменять данные в системе, следует ограничиться показом формы и сценария валидации без финальной успешной отправки.

Если допустима мутация тестовых данных, можно показать успешную отставку и наличие записи в `Strapi`.

Ожидаемый результат: показано, что прикладные формы встроены в систему и защищены server-side слоем.

7. Демонстрация публикационного контура

Время: 1 -- 2 минуты.

Что показать:

- наличие `managed webhook` в `Strapi`;
- подписку на события `entry.publish` и `entry.unpublish`;
- краткое объяснение цепочки `publish` → `webhook` → `rebuild`.

Что проговорить:

- локально подтвержден факт существования и синхронизации `webhook`;
- полный внешний `production`-путь с `Vercel` не требуется демонстрировать вживую;
- для целей ВКР важно показать сам инженерный контур и его воспроизводимость.

Ожидаемый результат: комиссия видит, что публикация контента связана с обновлением публичной витрины.

8. Итог и ограничения

Время: 1 минута.

Что проговорить в завершении:

- система реализует `CMS-first` модель для маркетингового сайта;
- контентное управление отделено от публичной витрины;
- реализованы локализация, `preview`, формы и публикационный контур;
- проект имеет воспроизводимый набор проверок и `acceptance`-сценариев.

Какие ограничения нужно проговаривать честно:

- английское наполнение `detail`-разделов `articles/projects` зависит от фактического `dataset`;
- текущий `SEO dataset` требует дополнительной финальной проверки индексации публичных страниц;
- `browser-level` аудит через `Lighthouse/axe` не является обязательной частью текущей `live`-демонстрации;
- `live`-показ внешнего `Vercel rebuild` не обязателен.

Что лучше не показывать как основной сценарий

Во время демонстрации не стоит делать центральным элементом показа:

- редактирование `production`-данных без заранее подготовленного тестового сценария;
- зависимые от внешней инфраструктуры действия, которые могут сорваться из-за сети;
- англоязычные `detail`-страницы `articles/projects`, если они не заполнены в текущем `dataset`;
- спорные `SEO`-утверждения без предварительно проверенного фактического состояния.

Резервный план

Если часть `live`-сценариев не открывается во время демонстрации, рекомендуется:

1. сохранить основную логику показа через уже работающие маршруты `/ru/`, `page`, `preview`, `vacancies`;
2. использовать заранее открытые вкладки браузера;
3. опираться на уже подготовленные диаграммы, `acceptance`-материалы и структуру сущностей в `Strapi`;
4. делать акцент на архитектуре, редакторском `workflow` и воспроизводимых проверках.

Вывод

Для предзащиты оптимален сценарий, в котором демонстрация строится не вокруг внесения новых изменений в проект, а вокруг уже реализованных и подтвержденных контуров: контентного управления, публичной витрины, preview, форм и публикации. Это позволяет показать зрелость решения и избежать риска live-ошибок, связанных с ненужной доработкой проекта непосредственно перед выступлением.